

A Byte of Python**The dir() function**

Â

Chapter 8. Modules

Â

The dir() function

You can use the built-in `dir` function to list the identifiers that a module defines. The identifiers are the functions, classes and variables defined in that module.

When you supply a module name to the `dir()` function, it returns the list of the names defined in that module. When no argument is applied to it, it returns the list of names defined in the current module.

Using the dir function

Example 8.4. Using the dir function

```
$ python
>>> import sys
>>> dir(sys) # get list of attributes for sys module
['__displayhook__', '__doc__', '__excepthook__', '__name__', '__stderr__',
 '__stdin__', '__stdout__', '_getframe', 'api_version', 'argv',
 'builtin_module_names', 'byteorder', 'call_tracing', 'callstats',
 'copyright', 'displayhook', 'exc_clear', 'exc_info', 'exc_type',
 'excepthook', 'exec_prefix', 'executable', 'exit', 'getcheckinterval',
 'getdefaultencoding', 'getdlopenflags', 'getfilesystemencoding',
 'getrecursionlimit', 'getrefcount', 'hexversion', 'maxint', 'maxunicode',
 'meta_path', 'modules', 'path', 'path_hooks', 'path_importer_cache',
 'platform', 'prefix', 'ps1', 'ps2', 'setcheckinterval', 'setdlopenflags',
 'setprofile', 'setrecursionlimit', 'settrace', 'stderr', 'stdin', 'stdout',
 'version', 'version_info', 'warnoptions']
>>> dir() # get list of attributes for current module
['__builtins__', '__doc__', '__name__', 'sys']
>>>
>>> a = 5 # create a new variable 'a'
>>> dir()
['__builtins__', '__doc__', '__name__', 'a', 'sys']
>>>
>>> del a # delete/remove a name
>>>
>>> dir()
['__builtins__', '__doc__', '__name__', 'sys']
>>>
```

How It Works

First, we see the usage of `dir` on the imported `sys` module. We can see the huge list of attributes that it contains.

Next, we use the `dir` function without passing parameters to it - by default, it returns the list of attributes for the current module. Notice that the list of imported modules is also part of this list.

In order to observe the `dir` in action, we define a new variable `a` and assign it a value and then check `dir` and we observe that there is an additional value in the list of the same name. We remove the variable/attribute of the current module using the `del` statement and the change is reflected again in the output of the `dir` function.

A note on `del` - this statement is used to *delete* a variable/name and after the statement has run, in this case `del a`, you can no longer access the variable `a` - it is as if it never existed before at all.



Â

Making your own ModulesÂ



Â

Â Summary